

<style>pre, code {white-space: pre-wrap !important; overflow-wrap: anywhere !important;} .jp-OutputArea-output, .jp-RenderedText {overflow: visible !important;} table {font-size: 10px !important;}</style>

```
In [1]: from pathlib import Path

DATA_PATH = Path(r"C:\Users\123wi\OneDrive\Desktop\duits uni\data analysis\complain")
print("CSV:", DATA_PATH)
```

CSV: C:\Users\123wi\OneDrive\Desktop\duits uni\data analysis\complaints.csv

Complaint preprocessing — full dataset, small batches

Here we prepare the complete complaint dataset without loading it all into RAM. We remove missing text, clean and tokenise it, lemmatise it, and create word counts and TF-IDF. **There is no stemming and no stop-word removal.**

Restart the kernel before using this version to release the old notebook's data. Save this notebook as `complaints_preprocessing.ipynb` beside the CSV. Run cells in order. The data are read in small pandas DataFrames and released after writing to disk.

Processed rows are stored as **JSONL**: one JSON object per line. Only original text, lemmatised text, a few identifiers and token counts are saved. Original text is useful for later BERTopic work. Token lists and intermediate text are not saved for every row. The word-count and TF-IDF matrices are saved as separate sparse batches.

1. Settings and libraries

Here we choose small CSV and feature batches. `MAX_SOURCE_ROWS = None` processes the whole CSV. An integer is only a first-N-rows test limit, not a random sample.

Lemmatisation uses **processes**, rather than Python threads. Each worker loads its own language resources. We start with at most four workers. Lower `N_WORKERS` to 2 if RAM is limited. More workers can improve throughput but also use more RAM; full CPU utilisation is not guaranteed. Batch sizes limit the number of rows, not the length of a single text.

```
In [2]: import csv
import gc
import importlib
import json
import math
import os
import sqlite3
import subprocess
import sys
import time
import uuid
```

```

from collections import Counter
from contextlib import closing
from datetime import datetime, timezone
from importlib.metadata import version
from itertools import islice

import nltk
import numpy as np
import pandas as pd
import psutil
from IPython.display import display
from scipy import sparse
from sklearn.preprocessing import normalize
from joblib.externals.loky import ProcessPoolExecutor

TEXT_COLUMN = "Consumer complaint narrative"
CHUNK_ROWS = 1_000
WORKER_BATCH_ROWS = 32
MATRIX_BATCH_ROWS = 1_000
MAX_SOURCE_ROWS = None

physical_cores = psutil.cpu_count(logical=False) or os.cpu_count() or 2
N_WORKERS = max(1, min(4, physical_cores - 1))

KEEP_ORIGINAL_TEXT = True
PRINT_EVERY_CHUNKS = 20

OUTPUT_ROOT = DATA_PATH.parent / "preprocessing_streamed"
NOTEBOOK_PATH = DATA_PATH.parent / "complaints_preprocessing.ipynb"
OUTPUT_ROOT.mkdir(parents=True, exist_ok=True)

pd.set_option("display.max_colwidth", 90)
pd.set_option("display.max_columns", 6)

package_versions = {
    package: version(package)
    for package in [
        "pandas", "numpy", "scipy", "scikit-learn",
        "nltk", "joblib", "psutil"
    ]
}

print("Python:", sys.version.split()[0])
print(
    "Workers:", N_WORKERS,
    "| CSV batch:", CHUNK_ROWS,
    "| Feature batch:", MATRIX_BATCH_ROWS
)
print("Source row limit:", MAX_SOURCE_ROWS, "(None means the complete CSV)")
print(
    "Available RAM now:",
    round(psutil.virtual_memory().available / 1024**3, 2),
    "GiB"
)
print(
    "Free output-disk space:",

```

```

round(psutil.disk_usage(str(OUTPUT_ROOT)).free / 1024**3, 1),
"GiB"
)
print("Stop-word removal: disabled. No vocabulary cap or frequency filtering.")

display(pd.Series(package_versions, name="Version").to_frame())

```

Python: 3.10.21

Workers: 4 | CSV batch: 1000 | Feature batch: 1000

Source row limit: None (None means the complete CSV)

Available RAM now: 19.81 GiB

Free output-disk space: 322.6 GiB

Stop-word removal: disabled. No vocabulary cap or frequency filtering.

	Version
pandas	2.3.3
numpy	2.2.6
scipy	1.15.3
scikit-learn	1.7.2
nltk	3.10.3
joblib	1.6.0
psutil	7.2.2

2. Prepare the language resources

Here we check the English POS tagger and WordNet. Missing resources are downloaded once before workers start. POS means part of speech, such as noun or verb. These English tools can make mistakes on ambiguous wording or non-English complaints.

```

In [3]: # Inspect paths without loading the models into the notebook process.
try:
    nltk.data.find("corpora/wordnet")
except LookupError:
    try:
        nltk.data.find("corpora/wordnet.zip")
    except LookupError:
        nltk.download("wordnet", raise_on_error=True)
try:
    nltk.data.find("taggers/averaged_perceptron_tagger_eng/")
except LookupError:
    nltk.download("averaged_perceptron_tagger_eng", raise_on_error=True)
print("NLTK resources are ready.")

```

NLTK resources are ready.

3. Cleaning, tokenisation and lemmatisation

Here we define the text-processing steps. We normalise Unicode, remove HTML tags, links, email addresses and redaction placeholders such as `XXXX`, and use lowercase. Tokenisation keeps words and contractions. Numbers and punctuation are not features.

Here we use POS-aware lemmatisation on the original tokens. Stop words and single-letter words are retained. A small cache avoids repeating dictionary lookups for the same word and POS. We write these functions to a Python helper so Windows workers can import them. This file is created automatically; no separate download is needed.

```
In [4]: WORKER_SOURCE = r'''
import html
import re
import unicodedata
from functools import lru_cache
import nltk
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import RegexpTokenizer

_TOKENIZER = RegexpTokenizer(r"^[^W\d_]+(?:'^[^W\d_]+)*")
_LEMMATISER = WordNetLemmatizer()
_POS = {"J": "a", "N": "n", "V": "v", "R": "r"}

def initialise_worker(data_paths):
    nltk.data.path[:] = data_paths
    nltk.corpus.wordnet.ensure_loaded()
    nltk.pos_tag(["ready"])

def clean_text(text):
    text = unicodedata.normalize("NFKC", html.unescape(text))
    text = text.replace("\u2019", "").replace("\u2018", "")
    text = re.sub(r"</?[A-Za-z][^>]*>", " ", text)
    text = re.sub(r"(?i)\b(?:https?://|www\.)\S+", " ", text)
    text = re.sub(r"(?i)\b[\w.-]+@[ \w.-]+\.[a-z]{2,}\b", " ", text)
    text = re.sub(r"\b{2,}\b", " ", text.lower())
    return re.sub(r"\s+", " ", text).strip()

@lru_cache(maxsize=10_000)
def lemma(word, pos):
    return _LEMMATISER.lemmatize(word, pos)

def process_batch(task):
    texts, show_examples = task
    cleaned = [clean_text(text) for text in texts]
    tokens = [_TOKENIZER.tokenize(text) for text in cleaned]
    tagged = nltk.pos_tag_sents(tokens)
    results = []
    for i, sentence in enumerate(tagged):
        lemmas = [lemma(word, _POS[tag[0]]) if tag[0] in _POS else word
                   for word, tag in sentence]
        item = {"lemmatised_text": " ".join(lemmas), "token_count": len(lemmas)}
        if show_examples and i < 3 and lemmas:
            item["example"] = {
                "original": texts[i][:400], "cleaned": cleaned[i][:400],
                "tokens": tokens[i][:15], "lemmas": lemmas[:15]
```

```

    }
    results.append(item)
    return results
...
WORKER_PATH = OUTPUT_ROOT / "_complaints_worker_v2.py"
WORKER_PATH.write_text(WORKER_SOURCE, encoding="utf-8")
if str(OUTPUT_ROOT) not in sys.path:
    sys.path.insert(0, str(OUTPUT_ROOT))
importlib.invalidate_caches()
import _complaints_worker_v2 as worker
worker = importlib.reload(worker)
print("Created the importable worker helper:", WORKER_PATH.name)

```

Created the importable worker helper: _complaints_worker_v2.py

4. Inspect only the file header

Here we load the header, not the dataset. Only the narrative, complaint ID, product and issue are read in the next step. Unneeded CSV columns never enter the working DataFrame.

```

In [5]: if not DATA_PATH.is_file():
        raise FileNotFoundError(f"Check DATA_PATH in the first cell: {DATA_PATH}")
source_columns = pd.read_csv(DATA_PATH, nrows=0, encoding="utf-8-sig").columns.tolist()
if TEXT_COLUMN not in source_columns:
    raise ValueError(f"Missing {TEXT_COLUMN!r}. Available columns: {source_columns}")
USE_COLUMNS = [name for name in [TEXT_COLUMN, "Complaint ID", "Product", "Issue"]
               if name in source_columns]
print("Source size:", round(DATA_PATH.stat().st_size / 1024**3, 2), "GiB")
print("Source columns:", len(source_columns))
print("Columns to read:", USE_COLUMNS)
if "Complaint ID" not in USE_COLUMNS:
    print("No complaint ID column: records will be retained without ID deduplicatio

```

Source size: 2.27 GiB

Source columns: 18

Columns to read: ['Consumer complaint narrative', 'Complaint ID', 'Product', 'Issue']

5. Stream the full CSV to JSONL

Here we remove null or blank narratives. We keep rows with missing optional metadata. Repeated, non-empty complaint IDs are removed across batches using a small disk database. Identical text under different IDs is kept because it may represent separate complaints. Rows without an ID are also kept. This does not identify repeated authors.

Each CSV chunk is processed by the worker pool, written to JSONL, and released. We save only three short before-and-after examples for display. Word counts and document frequencies are accumulated in SQLite on disk. A word contributes once to document frequency per complaint.

A new run folder is created each time, so old results are not overwritten. If interrupted, partial files are kept but are marked incomplete and cannot be used for vectorisation. This version restarts preprocessing rather than resuming an incomplete run.

```
In [6]: def open_database(path):
        connection = sqlite3.connect(path)
        connection.execute("PRAGMA cache_size = -16384") # About 16 MiB page cache.
        connection.execute("PRAGMA temp_store = FILE")
        return connection

def ram_gib():
    # Sum of resident memory of the kernel and workers; shared pages may be counted
    parent = psutil.Process()
    total = 0
    for process in [parent] + parent.children(recursive=True):
        try:
            total += process.memory_info().rss
        except (psutil.NoSuchProcess, psutil.AccessDenied):
            pass
    return total / 1024**3

def write_json(path, data):
    temporary = path.with_suffix(path.suffix + ".tmp")
    temporary.write_text(json.dumps(data, indent=2, ensure_ascii=False), encoding="utf-8")
    temporary.replace(path)

def iter_jsonl_batches(path, batch_rows):
    with path.open(encoding="utf-8") as handle:
        while True:
            batch = list(islice(handle, batch_rows))
            if not batch:
                break
            yield [json.loads(line) for line in batch]

print("Streaming helpers are ready.")
```

Streaming helpers are ready.

```
In [7]: def run_preprocessing(data_path, output_root, use_columns, workers, chunk_rows,
                             worker_batch_rows, max_source_rows=None, keep_original=True):
    if workers < 1 or chunk_rows < 1 or worker_batch_rows < 1:
        raise ValueError("Worker and batch counts must be positive.")
    if max_source_rows is not None and max_source_rows < 1:
        raise ValueError("MAX_SOURCE_ROWS must be positive or None.")
    stamp = datetime.now(timezone.utc).strftime("%Y%m%d_%H%M%S")
    run_dir = output_root / f"run_{stamp}_{uuid.uuid4().hex[:6]}"
    run_dir.mkdir(parents=True)
    source_stat = data_path.stat()
    stats = dict(source_rows=0, missing_or_blank=0, duplicate_ids=0,
                 empty_after_cleaning=0, kept_rows=0, total_tokens=0)
    info = {"status": "running", "source": str(data_path),
            "source_bytes": source_stat.st_size, "source_mtime_ns": source_stat.st_mtime_ns,
            "workers": workers, "chunk_rows": chunk_rows, "worker_batch_rows": worker_batch_rows,
            "max_source_rows": max_source_rows, "keep_original_text": keep_original,
            "stop_words_removed": False, "stemming": False,
```

```

        "python": sys.version.split()[0], "packages": package_versions,
        "created_utc": datetime.now(timezone.utc).isoformat()}
write_json(run_dir / "preprocessing_status.json", info)
connection = open_database(run_dir / "statistics.sqlite")
connection.execute("CREATE TABLE seen_ids (id TEXT PRIMARY KEY) WITHOUT ROWID")
connection.execute("CREATE TABLE terms (term TEXT PRIMARY KEY, doc_freq INTEGER
                    "occurrences INTEGER) WITHOUT ROWID")

executor = None
failed = True
examples = []
start = time.perf_counter()
peak_observed_ram = ram_gib()
print("Run folder:", run_dir)
print("Reading small chunks. Workers load language models on their first task.")
progress = display("Starting preprocessing...", display_id=True)
try:
    # An owned pool is explicitly shut down at the end, releasing worker RAM.
    executor = ProcessPoolExecutor(
        max_workers=workers, initializer=worker.initialise_worker,
        initargs=(list(nltk.data.path),),
        env={"OMP_NUM_THREADS": "1", "OPENBLAS_NUM_THREADS": "1", "MKL_NUM_THRE
    )
    with pd.read_csv(data_path, usecols=use_columns, dtype=str,
                     keep_default_na=False, encoding="utf-8-sig",
                     chunksize=chunk_rows, nrows=max_source_rows) as reader,
        for chunk_number, chunk in enumerate(reader, start=1):
            records = []
            for values in chunk.itertuples(index=False, name=None):
                source = dict(zip(chunk.columns, values))
                stats["source_rows"] += 1
                text = source[TEXT_COLUMN]
                if not isinstance(text, str) or not text.strip():
                    stats["missing_or_blank"] += 1
                    continue
                complaint_id = source.get("Complaint ID", "").strip()
                if complaint_id:
                    inserted = connection.execute(
                        "INSERT OR IGNORE INTO seen_ids VALUES (?)", (complaint
                    ).rowcount
                    if inserted == 0:
                        stats["duplicate_ids"] += 1
                        continue
                records.append({"source_record": stats["source_rows"],
                               "complaint_id": complaint_id or None,
                               "product": source.get("Product") or None,
                               "issue": source.get("Issue") or None,
                               "original_text": text})

            del chunk
            # Only this CSV chunk is queued. Results are yielded in input order
            tasks = ([record["original_text"] for record in
                      records[i:i + worker_batch_rows]], len(examples) < 3)
            for i in range(0, len(records), worker_batch_rows)
            results = executor.map(worker.process_batch, tasks, chunksize=1)
            local_df, local_counts = Counter(), Counter()
            position = 0
            for result_batch in results:

```

```

        for result in result_batch:
            record = records[position]
            position += 1
            example = result.pop("example", None)
            if result["token_count"] == 0:
                stats["empty_after_cleaning"] += 1
                continue
            if example and len(examples) < 3:
                examples.append(example)
            frequencies = Counter(result["lemmatised_text"].split())
            local_counts.update(frequencies)
            local_df.update(frequencies.keys())
            record.update(result)
            record["matrix_row"] = stats["kept_rows"]
            if not keep_original:
                record.pop("original_text")
            output.write(json.dumps(record, ensure_ascii=False) + "\n")
            stats["kept_rows"] += 1
            stats["total_tokens"] += record["token_count"]
    assert position == len(records)
    connection.executemany(
        "INSERT INTO terms VALUES (?, ?, ?) ON CONFLICT(term) DO UPDATE "
        "doc_freq=doc_freq+excluded.doc_freq, "
        "occurrences=occurrences+excluded.occurrences",
        ((term, frequency, local_counts[term]) for term, frequency in 1
    )
    connection.commit()
    output.flush()
    memory = ram_gib()
    peak_observed_ram = max(peak_observed_ram, memory)
    elapsed = time.perf_counter() - start
    message = (f"Read {stats['source_rows']:,} | kept {stats['kept_rows']:,} "
              f"{elapsed / 60:.1f} min | kernel + workers RSS ~{memory} GB")
    progress.update(message)
    if chunk_number == 1 or chunk_number % PRINT_EVERY_CHUNKS == 0:
        print(message)
    write_json(run_dir / "preprocessing_status.json",
              dict(info, stats=stats, elapsed_seconds=elapsed))
    # No full dataset, token columns, or result list survives a chunk.
    del records, results, tasks, local_df, local_counts
    gc.collect()
    assert stats["source_rows"] == sum(stats[key] for key in
        ["missing_or_blank", "duplicate_ids", "empty_after_cleaning", "kept_row"])
    if stats["kept_rows"] == 0:
        raise ValueError("No usable complaints remain. Inspect the source text.")
    if data_path.stat().st_mtime_ns != source_stat.st_mtime_ns:
        raise RuntimeError("The source CSV changed during this run. Use a stable source")
    (run_dir / "complaints.partial.jsonl").replace(run_dir / "complaints.jsonl")
    write_json(run_dir / "examples.json", examples)
    info.update(status="complete", stats=stats,
              elapsed_seconds=time.perf_counter() - start,
              observed_peak_process_rss_gib=peak_observed_ram)
    write_json(run_dir / "preprocessing_status.json", info)
    failed = False
except BaseException as error:
    info.update(status="incomplete", stats=stats,

```

```

        error=f"{type(error).__name__}: {error}")
    write_json(run_dir / "preprocessing_status.json", info)
    raise
finally:
    connection.close()
    if executor is not None:
        executor.shutdown(wait=True, kill_workers=failed)
    gc.collect()
print("Preprocessing complete. Worker processes have been closed.")
print("Observed peak RSS is sampled at chunk boundaries, not a guaranteed maximum")
return run_dir

print("Full-dataset preprocessing function is ready.")

```

Full-dataset preprocessing function is ready.

```

In [8]: RUN_DIR = run_preprocessing(
        DATA_PATH, OUTPUT_ROOT, USE_COLUMNS, N_WORKERS, CHUNK_ROWS,
        WORKER_BATCH_ROWS, MAX_SOURCE_ROWS, KEEP_ORIGINAL_TEXT
    )
    OUTPUT_DIR = RUN_DIR # Also used by the final PDF export cell.
    preprocessing_info = json.loads((RUN_DIR / "preprocessing_status.json").read_text())
    display(pd.Series(preprocessing_info["stats"], name="Rows / tokens").to_frame())
    print("Processed JSONL:", RUN_DIR / "complaints.jsonl")
    print("Elapsed minutes:", round(preprocessing_info["elapsed_seconds"] / 60, 2))
    print("Observed peak process RSS:",
          round(preprocessing_info["observed_peak_process_rss_gib"], 2), "GiB")
    print("RSS after workers closed:", round(ram_gib(), 2), "GiB")

```

Run folder: C:\Users\123wi\OneDrive\Desktop\duits uni\data analysis\preprocessing_status\run_20260923_103853_ac056b

Reading small chunks. Workers load language models on their first task.

'Read 3,585,952 | kept 1,292,206 | 46.1 min | kernel + workers RSS ~1.36 GiB'

Read 1,000		kept 26		0.1 min		kernel + workers RSS ~1.32 GiB
Read 20,000		kept 1,717		0.2 min		kernel + workers RSS ~1.32 GiB
Read 40,000		kept 6,737		0.4 min		kernel + workers RSS ~1.33 GiB
Read 60,000		kept 13,700		0.6 min		kernel + workers RSS ~1.33 GiB
Read 80,000		kept 20,877		0.9 min		kernel + workers RSS ~1.33 GiB
Read 100,000		kept 28,804		1.2 min		kernel + workers RSS ~1.33 GiB
Read 120,000		kept 35,638		1.4 min		kernel + workers RSS ~1.33 GiB
Read 140,000		kept 42,606		1.7 min		kernel + workers RSS ~1.33 GiB
Read 160,000		kept 50,003		1.9 min		kernel + workers RSS ~1.33 GiB
Read 180,000		kept 56,459		2.1 min		kernel + workers RSS ~1.33 GiB
Read 200,000		kept 62,984		2.4 min		kernel + workers RSS ~1.33 GiB
Read 220,000		kept 69,491		2.6 min		kernel + workers RSS ~1.33 GiB
Read 240,000		kept 75,844		2.9 min		kernel + workers RSS ~1.34 GiB
Read 260,000		kept 83,339		3.1 min		kernel + workers RSS ~1.34 GiB
Read 280,000		kept 91,045		3.4 min		kernel + workers RSS ~1.34 GiB
Read 300,000		kept 99,067		3.7 min		kernel + workers RSS ~1.34 GiB
Read 320,000		kept 107,251		4.0 min		kernel + workers RSS ~1.34 GiB
Read 340,000		kept 115,629		4.2 min		kernel + workers RSS ~1.34 GiB
Read 360,000		kept 124,273		4.6 min		kernel + workers RSS ~1.34 GiB
Read 380,000		kept 133,268		4.9 min		kernel + workers RSS ~1.34 GiB
Read 400,000		kept 142,255		5.2 min		kernel + workers RSS ~1.34 GiB
Read 420,000		kept 151,197		5.5 min		kernel + workers RSS ~1.34 GiB
Read 440,000		kept 160,139		5.8 min		kernel + workers RSS ~1.34 GiB
Read 460,000		kept 168,241		6.1 min		kernel + workers RSS ~1.34 GiB
Read 480,000		kept 175,474		6.3 min		kernel + workers RSS ~1.34 GiB
Read 500,000		kept 183,837		6.6 min		kernel + workers RSS ~1.34 GiB
Read 520,000		kept 193,538		6.9 min		kernel + workers RSS ~1.34 GiB
Read 540,000		kept 203,278		7.2 min		kernel + workers RSS ~1.34 GiB
Read 560,000		kept 212,995		7.5 min		kernel + workers RSS ~1.34 GiB
Read 580,000		kept 222,446		7.8 min		kernel + workers RSS ~1.34 GiB
Read 600,000		kept 232,338		8.1 min		kernel + workers RSS ~1.34 GiB
Read 620,000		kept 241,793		8.4 min		kernel + workers RSS ~1.34 GiB
Read 640,000		kept 251,177		8.7 min		kernel + workers RSS ~1.34 GiB
Read 660,000		kept 261,394		9.0 min		kernel + workers RSS ~1.34 GiB
Read 680,000		kept 272,071		9.4 min		kernel + workers RSS ~1.34 GiB
Read 700,000		kept 282,091		9.7 min		kernel + workers RSS ~1.34 GiB
Read 720,000		kept 292,203		10.0 min		kernel + workers RSS ~1.34 GiB
Read 740,000		kept 302,688		10.3 min		kernel + workers RSS ~1.34 GiB
Read 760,000		kept 312,650		10.6 min		kernel + workers RSS ~1.34 GiB
Read 780,000		kept 323,250		10.9 min		kernel + workers RSS ~1.34 GiB
Read 800,000		kept 333,836		11.3 min		kernel + workers RSS ~1.34 GiB
Read 820,000		kept 343,864		11.6 min		kernel + workers RSS ~1.34 GiB
Read 840,000		kept 352,813		11.8 min		kernel + workers RSS ~1.34 GiB
Read 860,000		kept 361,938		12.1 min		kernel + workers RSS ~1.34 GiB
Read 880,000		kept 371,582		12.4 min		kernel + workers RSS ~1.34 GiB
Read 900,000		kept 380,130		12.7 min		kernel + workers RSS ~1.34 GiB
Read 920,000		kept 388,992		13.0 min		kernel + workers RSS ~1.34 GiB
Read 940,000		kept 398,352		13.3 min		kernel + workers RSS ~1.35 GiB
Read 960,000		kept 408,032		13.6 min		kernel + workers RSS ~1.35 GiB
Read 980,000		kept 417,916		13.9 min		kernel + workers RSS ~1.35 GiB
Read 1,000,000		kept 428,320		14.2 min		kernel + workers RSS ~1.35 GiB
Read 1,020,000		kept 437,949		14.5 min		kernel + workers RSS ~1.35 GiB
Read 1,040,000		kept 448,107		14.8 min		kernel + workers RSS ~1.35 GiB
Read 1,060,000		kept 457,449		15.1 min		kernel + workers RSS ~1.35 GiB
Read 1,080,000		kept 465,973		15.4 min		kernel + workers RSS ~1.35 GiB
Read 1,100,000		kept 473,954		15.6 min		kernel + workers RSS ~1.35 GiB

Read 1,120,000	kept 481,271	15.9 min	kernel + workers	RSS ~1.35 GiB
Read 1,140,000	kept 489,346	16.1 min	kernel + workers	RSS ~1.35 GiB
Read 1,160,000	kept 498,246	16.4 min	kernel + workers	RSS ~1.35 GiB
Read 1,180,000	kept 506,238	16.7 min	kernel + workers	RSS ~1.35 GiB
Read 1,200,000	kept 514,710	17.0 min	kernel + workers	RSS ~1.35 GiB
Read 1,220,000	kept 522,904	17.3 min	kernel + workers	RSS ~1.35 GiB
Read 1,240,000	kept 530,325	17.5 min	kernel + workers	RSS ~1.35 GiB
Read 1,260,000	kept 538,233	17.8 min	kernel + workers	RSS ~1.35 GiB
Read 1,280,000	kept 546,272	18.1 min	kernel + workers	RSS ~1.35 GiB
Read 1,300,000	kept 556,107	18.4 min	kernel + workers	RSS ~1.35 GiB
Read 1,320,000	kept 565,262	18.7 min	kernel + workers	RSS ~1.35 GiB
Read 1,340,000	kept 572,841	18.9 min	kernel + workers	RSS ~1.35 GiB
Read 1,360,000	kept 580,758	19.2 min	kernel + workers	RSS ~1.35 GiB
Read 1,380,000	kept 588,206	19.5 min	kernel + workers	RSS ~1.35 GiB
Read 1,400,000	kept 595,499	19.7 min	kernel + workers	RSS ~1.35 GiB
Read 1,420,000	kept 602,776	20.0 min	kernel + workers	RSS ~1.35 GiB
Read 1,440,000	kept 609,553	20.2 min	kernel + workers	RSS ~1.35 GiB
Read 1,460,000	kept 615,875	20.4 min	kernel + workers	RSS ~1.35 GiB
Read 1,480,000	kept 622,337	20.7 min	kernel + workers	RSS ~1.35 GiB
Read 1,500,000	kept 628,927	20.9 min	kernel + workers	RSS ~1.35 GiB
Read 1,520,000	kept 635,486	21.2 min	kernel + workers	RSS ~1.35 GiB
Read 1,540,000	kept 641,959	21.4 min	kernel + workers	RSS ~1.35 GiB
Read 1,560,000	kept 648,413	21.6 min	kernel + workers	RSS ~1.35 GiB
Read 1,580,000	kept 655,345	21.9 min	kernel + workers	RSS ~1.35 GiB
Read 1,600,000	kept 661,922	22.1 min	kernel + workers	RSS ~1.35 GiB
Read 1,620,000	kept 667,771	22.3 min	kernel + workers	RSS ~1.35 GiB
Read 1,640,000	kept 674,138	22.5 min	kernel + workers	RSS ~1.35 GiB
Read 1,660,000	kept 680,558	22.8 min	kernel + workers	RSS ~1.35 GiB
Read 1,680,000	kept 686,214	23.0 min	kernel + workers	RSS ~1.35 GiB
Read 1,700,000	kept 692,349	23.2 min	kernel + workers	RSS ~1.35 GiB
Read 1,720,000	kept 698,497	23.4 min	kernel + workers	RSS ~1.35 GiB
Read 1,740,000	kept 705,170	23.7 min	kernel + workers	RSS ~1.35 GiB
Read 1,760,000	kept 711,633	23.9 min	kernel + workers	RSS ~1.35 GiB
Read 1,780,000	kept 717,942	24.1 min	kernel + workers	RSS ~1.35 GiB
Read 1,800,000	kept 724,551	24.4 min	kernel + workers	RSS ~1.35 GiB
Read 1,820,000	kept 731,593	24.6 min	kernel + workers	RSS ~1.35 GiB
Read 1,840,000	kept 738,921	24.9 min	kernel + workers	RSS ~1.35 GiB
Read 1,860,000	kept 745,789	25.2 min	kernel + workers	RSS ~1.35 GiB
Read 1,880,000	kept 753,155	25.4 min	kernel + workers	RSS ~1.35 GiB
Read 1,900,000	kept 760,584	25.7 min	kernel + workers	RSS ~1.35 GiB
Read 1,920,000	kept 767,504	25.9 min	kernel + workers	RSS ~1.35 GiB
Read 1,940,000	kept 774,269	26.2 min	kernel + workers	RSS ~1.35 GiB
Read 1,960,000	kept 780,961	26.4 min	kernel + workers	RSS ~1.35 GiB
Read 1,980,000	kept 787,729	26.6 min	kernel + workers	RSS ~1.35 GiB
Read 2,000,000	kept 795,149	26.9 min	kernel + workers	RSS ~1.35 GiB
Read 2,020,000	kept 802,579	27.1 min	kernel + workers	RSS ~1.35 GiB
Read 2,040,000	kept 809,638	27.4 min	kernel + workers	RSS ~1.35 GiB
Read 2,060,000	kept 816,198	27.6 min	kernel + workers	RSS ~1.35 GiB
Read 2,080,000	kept 823,298	27.9 min	kernel + workers	RSS ~1.35 GiB
Read 2,100,000	kept 830,498	28.1 min	kernel + workers	RSS ~1.35 GiB
Read 2,120,000	kept 837,534	28.4 min	kernel + workers	RSS ~1.35 GiB
Read 2,140,000	kept 845,111	28.7 min	kernel + workers	RSS ~1.35 GiB
Read 2,160,000	kept 852,227	28.9 min	kernel + workers	RSS ~1.35 GiB
Read 2,180,000	kept 859,866	29.2 min	kernel + workers	RSS ~1.35 GiB
Read 2,200,000	kept 867,472	29.4 min	kernel + workers	RSS ~1.35 GiB
Read 2,220,000	kept 874,187	29.7 min	kernel + workers	RSS ~1.35 GiB

Read 2,240,000	kept 881,005	29.9 min	kernel + workers RSS ~1.35 GiB
Read 2,260,000	kept 887,884	30.2 min	kernel + workers RSS ~1.36 GiB
Read 2,280,000	kept 894,998	30.4 min	kernel + workers RSS ~1.36 GiB
Read 2,300,000	kept 902,124	30.7 min	kernel + workers RSS ~1.36 GiB
Read 2,320,000	kept 909,344	30.9 min	kernel + workers RSS ~1.36 GiB
Read 2,340,000	kept 916,634	31.1 min	kernel + workers RSS ~1.36 GiB
Read 2,360,000	kept 923,619	31.4 min	kernel + workers RSS ~1.36 GiB
Read 2,380,000	kept 930,345	31.6 min	kernel + workers RSS ~1.36 GiB
Read 2,400,000	kept 937,240	31.9 min	kernel + workers RSS ~1.36 GiB
Read 2,420,000	kept 943,545	32.1 min	kernel + workers RSS ~1.36 GiB
Read 2,440,000	kept 950,100	32.4 min	kernel + workers RSS ~1.36 GiB
Read 2,460,000	kept 956,309	32.6 min	kernel + workers RSS ~1.36 GiB
Read 2,480,000	kept 962,790	32.8 min	kernel + workers RSS ~1.36 GiB
Read 2,500,000	kept 968,433	33.1 min	kernel + workers RSS ~1.36 GiB
Read 2,520,000	kept 974,636	33.3 min	kernel + workers RSS ~1.36 GiB
Read 2,540,000	kept 979,734	33.5 min	kernel + workers RSS ~1.36 GiB
Read 2,560,000	kept 985,784	33.7 min	kernel + workers RSS ~1.36 GiB
Read 2,580,000	kept 991,859	34.0 min	kernel + workers RSS ~1.36 GiB
Read 2,600,000	kept 998,060	34.2 min	kernel + workers RSS ~1.36 GiB
Read 2,620,000	kept 1,004,426	34.4 min	kernel + workers RSS ~1.36 GiB
Read 2,640,000	kept 1,011,032	34.7 min	kernel + workers RSS ~1.36 GiB
Read 2,660,000	kept 1,017,714	34.9 min	kernel + workers RSS ~1.36 GiB
Read 2,680,000	kept 1,024,390	35.2 min	kernel + workers RSS ~1.36 GiB
Read 2,700,000	kept 1,030,619	35.4 min	kernel + workers RSS ~1.36 GiB
Read 2,720,000	kept 1,036,919	35.6 min	kernel + workers RSS ~1.36 GiB
Read 2,740,000	kept 1,043,980	35.9 min	kernel + workers RSS ~1.36 GiB
Read 2,760,000	kept 1,050,466	36.2 min	kernel + workers RSS ~1.36 GiB
Read 2,780,000	kept 1,056,943	36.4 min	kernel + workers RSS ~1.36 GiB
Read 2,800,000	kept 1,062,992	36.7 min	kernel + workers RSS ~1.36 GiB
Read 2,820,000	kept 1,069,251	36.9 min	kernel + workers RSS ~1.36 GiB
Read 2,840,000	kept 1,076,184	37.2 min	kernel + workers RSS ~1.36 GiB
Read 2,860,000	kept 1,082,426	37.4 min	kernel + workers RSS ~1.36 GiB
Read 2,880,000	kept 1,090,204	37.7 min	kernel + workers RSS ~1.36 GiB
Read 2,900,000	kept 1,097,742	38.0 min	kernel + workers RSS ~1.36 GiB
Read 2,920,000	kept 1,107,204	38.3 min	kernel + workers RSS ~1.36 GiB
Read 2,940,000	kept 1,117,205	38.6 min	kernel + workers RSS ~1.36 GiB
Read 2,960,000	kept 1,127,979	39.0 min	kernel + workers RSS ~1.36 GiB
Read 2,980,000	kept 1,139,414	39.4 min	kernel + workers RSS ~1.36 GiB
Read 3,000,000	kept 1,147,856	39.6 min	kernel + workers RSS ~1.36 GiB
Read 3,020,000	kept 1,151,692	39.8 min	kernel + workers RSS ~1.36 GiB
Read 3,040,000	kept 1,158,096	40.1 min	kernel + workers RSS ~1.36 GiB
Read 3,060,000	kept 1,165,299	40.3 min	kernel + workers RSS ~1.36 GiB
Read 3,080,000	kept 1,172,442	40.6 min	kernel + workers RSS ~1.36 GiB
Read 3,100,000	kept 1,178,344	40.8 min	kernel + workers RSS ~1.36 GiB
Read 3,120,000	kept 1,181,632	41.0 min	kernel + workers RSS ~1.36 GiB
Read 3,140,000	kept 1,185,807	41.2 min	kernel + workers RSS ~1.36 GiB
Read 3,160,000	kept 1,189,520	41.3 min	kernel + workers RSS ~1.36 GiB
Read 3,180,000	kept 1,193,171	41.5 min	kernel + workers RSS ~1.36 GiB
Read 3,200,000	kept 1,195,164	41.6 min	kernel + workers RSS ~1.36 GiB
Read 3,220,000	kept 1,198,523	41.8 min	kernel + workers RSS ~1.36 GiB
Read 3,240,000	kept 1,201,977	41.9 min	kernel + workers RSS ~1.36 GiB
Read 3,260,000	kept 1,206,192	42.1 min	kernel + workers RSS ~1.36 GiB
Read 3,280,000	kept 1,210,199	42.3 min	kernel + workers RSS ~1.36 GiB
Read 3,300,000	kept 1,214,689	42.5 min	kernel + workers RSS ~1.36 GiB
Read 3,320,000	kept 1,218,770	42.7 min	kernel + workers RSS ~1.36 GiB
Read 3,340,000	kept 1,226,159	42.9 min	kernel + workers RSS ~1.36 GiB

Read 3,360,000 | kept 1,232,065 | 43.2 min | kernel + workers RSS ~1.36 GiB
 Read 3,380,000 | kept 1,238,073 | 43.4 min | kernel + workers RSS ~1.36 GiB
 Read 3,400,000 | kept 1,243,690 | 43.6 min | kernel + workers RSS ~1.36 GiB
 Read 3,420,000 | kept 1,248,890 | 43.8 min | kernel + workers RSS ~1.36 GiB
 Read 3,440,000 | kept 1,255,873 | 44.1 min | kernel + workers RSS ~1.36 GiB
 Read 3,460,000 | kept 1,260,396 | 44.3 min | kernel + workers RSS ~1.36 GiB
 Read 3,480,000 | kept 1,268,850 | 44.6 min | kernel + workers RSS ~1.36 GiB
 Read 3,500,000 | kept 1,276,645 | 44.9 min | kernel + workers RSS ~1.36 GiB
 Read 3,520,000 | kept 1,284,781 | 45.2 min | kernel + workers RSS ~1.36 GiB
 Read 3,540,000 | kept 1,288,659 | 45.4 min | kernel + workers RSS ~1.36 GiB
 Read 3,560,000 | kept 1,289,430 | 45.6 min | kernel + workers RSS ~1.36 GiB
 Read 3,580,000 | kept 1,291,402 | 45.9 min | kernel + workers RSS ~1.36 GiB
 Preprocessing complete. Worker processes have been closed.
 Observed peak RSS is sampled at chunk boundaries, not a guaranteed maximum.

Rows / tokens	
source_rows	3585952
missing_or_blank	2293687
duplicate_ids	0
empty_after_cleaning	59
kept_rows	1292206
total_tokens	215639113

Processed JSONL: C:\Users\123wi\OneDrive\Desktop\duits uni\data analysis\preprocessing_streamed\run_20260923_103853_ac056b\complaints.jsonl
 Elapsed minutes: 46.06
 Observed peak process RSS: 1.36 GiB
 RSS after workers closed: 0.22 GiB

6. Show small pandas previews

Here we show original text, cleaned text, tokens and lemmas for three complaints. The `df` variable contains only five processed records, not the full dataset. Intermediate previews are deleted once displayed. The saved original narrative is not truncated.

```
In [9]: examples = json.loads((RUN_DIR / "examples.json").read_text(encoding="utf-8"))
for number, example in enumerate(examples, start=1):
    print(f"Example {number} (short display only)")
    display(pd.DataFrame({"Stage": ["Original", "Cleaned", "Tokens", "Lemmas"],
                           "Text": [example["original"], example["cleaned"],
                                     str(example["tokens"]), str(example["lemmas"])]}))
del examples

with (RUN_DIR / "complaints.jsonl").open(encoding="utf-8") as handle:
    df = pd.DataFrame(json.loads(line) for line in islice(handle, 5))
    display(df[["matrix_row", "complaint_id", "token_count", "lemmatised_text"]])
    print("df holds only", len(df), "preview rows.")
with closing(open_database(RUN_DIR / "statistics.sqlite")) as connection:
    common_words = pd.read_sql_query(
```

```

        "SELECT term, occurrences, doc_freq FROM terms ORDER BY occurrences DESC, t
        connection
    )
display(common_words)
del common_words
gc.collect();

```

Example 1 (short display only)

	Stage	Text
0	Original	My minimum payment for my Fortiva Retail Credit Card is {\$63.00} a month. However ever...
1	Cleaned	my minimum payment for my fortiva retail credit card is {\$63.00} a month. however ever...
2	Tokens	['my', 'minimum', 'payment', 'for', 'my', 'fortiva', 'retail', 'credit', 'card', 'is', ...]
3	Lemmas	['my', 'minimum', 'payment', 'for', 'my', 'fortiva', 'retail', 'credit', 'card', 'be', ...]

Example 2 (short display only)

	Stage	Text
0	Original	I HAD MY IDENTITY STOLEN AND SOMEONE CREATED FRAUDELUNET ACCOUNTS UNDER MY NAME SO IM ...
1	Cleaned	i had my identity stolen and someone created fraudelunet accounts under my name so im ...
2	Tokens	['i', 'had', 'my', 'identity', 'stolen', 'and', 'someone', 'created', 'fraudelunet', '...
3	Lemmas	['i', 'have', 'my', 'identity', 'steal', 'and', 'someone', 'create', 'fraudelunet', 'a...

Example 3 (short display only)

	Stage	Text
0	Original	I'm sending this complaint to inform AGAIN the credit bureau that I was a victim of Id...
1	Cleaned	i'm sending this complaint to inform again the credit bureau that i was a victim of id...
2	Tokens	['i'm', 'sending', 'this', 'complaint', 'to', 'inform', 'again', 'the', 'credit', 'bur...
3	Lemmas	['i'm', 'send', 'this', 'complaint', 'to', 'inform', 'again', 'the', 'credit', 'bureau...

	matrix_row	complaint_id	token_count	lemmatised_text
0	0	6854957	144	my minimum payment for my fortiva retail credit card be a month however every time i g...
1	1	6840809	82	i have my identity steal and someone create fraudelunet account under my name so im tr...
2	2	6823436	67	i'm send this complaint to inform again the credit bureau that i be a victim of identi...
3	3	6835773	621	identity theft florida date of birth s florida experian tx pa re last of social securi...
4	4	6821760	194	i have file numerous complaint with the credit bureau s about my state farm credit car...

df holds only 5 preview rows.

	term	occurrences	doc_freq
0	the	9598183	1132384
1	be	9232539	1174256
2	i	7833378	1161348
3	to	7731529	1152344
4	and	6152781	1105598
5	a	4935592	1044335
6	my	4926982	1126244
7	of	4238494	931135
8	have	4059483	1038238
9	that	3409572	881491
10	in	2666187	846753
11	on	2660597	903693
12	this	2579964	817858
13	account	2427416	734185
14	not	2420817	910964

7. Build one vocabulary and global IDF

Here we assign one feature number to each word. Every observed lemmatised word is kept, including stop words. Vocabulary and IDF values are stored in SQLite and a CSV on disk. Only small groups of vocabulary rows are held in RAM.

IDF is calculated **once from all retained complaints**, not separately inside each batch:

```
idf = log((1 + number_of_complaints) / (1 + document_frequency)) + 1
```

This is the smoothed IDF used by scikit-learn. Later batches share exactly the same feature indices and IDF values. They remain comparable across the full dataset.

```
In [10]: def build_vocabulary(run_dir):
    info = json.loads((run_dir / "preprocessing_status.json").read_text(encoding="u
    if info["status"] != "complete":
        raise RuntimeError("Preprocessing is incomplete.")
    document_count = info["stats"]["kept_rows"]
    status_path = run_dir / "vocabulary_status.json"
    write_json(status_path, {"status": "running"})
    connection = None
    feature_count = 0
    try:
        connection = open_database(run_dir / "statistics.sqlite")
        connection.execute("DROP TABLE IF EXISTS features")
        connection.execute("CREATE TABLE features (feature_index INTEGER PRIMARY KE
            "term TEXT UNIQUE, doc_freq INTEGER, occurrences INTEGER
        cursor = connection.execute("SELECT term, doc_freq, occurrences FROM terms
        with (run_dir / "vocabulary.partial.csv").open("w", newline="", encoding="u
        writer = csv.writer(handle)
        writer.writerow(["feature_index", "term", "document_frequency", "occurr
        while True:
            rows = cursor.fetchmany(2_000)
            if not rows:
                break
            feature_rows = [
                (feature_count + i, term, frequency, occurrences,
                 math.log((1 + document_count) / (1 + frequency)) + 1)
                for i, (term, frequency, occurrences) in enumerate(rows)
            ]
            connection.executemany("INSERT INTO features VALUES (?, ?, ?, ?, ?)
            writer.writerows(feature_rows)
            feature_count += len(rows)
        connection.commit()
        (run_dir / "vocabulary.partial.csv").replace(run_dir / "vocabulary.csv")
    except BaseException as error:
        write_json(status_path, {"status": "incomplete",
                                "error": f"{type(error).__name__}: {error}"})
        raise
    finally:
        if connection is not None:
            connection.close()
    write_json(status_path,
        {"status": "complete", "documents": document_count, "features": feat
        "ordering": "alphabetical", "stop_words_removed": False,
        "frequency_filtering": False, "smooth_idf": True})
    print(f"Vocabulary complete: {feature_count:,} features from {document_count:,}")
    return feature_count

FEATURE_COUNT = build_vocabulary(RUN_DIR)
with closing(open_database(RUN_DIR / "statistics.sqlite")) as connection:
```

```
display(pd.read_sql_query("SELECT * FROM features ORDER BY feature_index LIMIT 10"))
print("The complete vocabulary stays on disk.")
```

Vocabulary complete: 168,291 features from 1,292,206 complaints.

	feature_index	term	doc_freq	occurrences	idf
0	0	a	1044335	4935592	1.212970
1	1	a'a	1	1	14.378715
2	2	a'am	2	2	13.973250
3	3	a'ed	3	3	13.685568
4	4	a'rent	1	1	14.378715
5	5	a'taketh	1	1	14.378715
6	6	aa	462	641	8.934135
7	7	aaa	101	223	10.446889
8	8	aaaa	2	2	13.973250
9	9	aaaaa	1	1	14.378715

The complete vocabulary stays on disk.

8. Save word counts and TF-IDF in sparse batches

Here we reread a small JSONL batch, look up its words in the disk vocabulary, and build sparse matrices. We multiply each count by its global IDF and L2-normalise each TF-IDF row. We save the matrices before reading the next batch. No full matrix or feature DataFrame is created. TF-IDF uses float32 to reduce storage, with minor rounding compared with float64.

A manifest records each batch's filenames and row range. Its `row_start` is inclusive and `row_end` exclusive. These indices match `matrix_row` in the JSONL file.

```
In [11]: def matrices_for_batch(records, connection, feature_count):
# These counters contain only this batch, never the whole corpus.
counters = [Counter(record["lemmatised_text"].split()) for record in records]
words = list({word for frequencies in counters for word in frequencies})
lookup = {}
for offset in range(0, len(words), 500):
    group = words[offset:offset + 500]
    placeholders = ",".join("{}" for _ in group)
    for word, feature_index, idf in connection.execute(
        f"SELECT term, feature_index, idf FROM features WHERE term IN ({placeholders})"):
        lookup[word] = (feature_index, idf)
if len(lookup) != len(words):
    raise ValueError("A batch word is missing from the global vocabulary.")
values, indices, weights, pointers = [], [], [], [0]
for frequencies in counters:
    for word, count in frequencies.items():
```

```

        feature_index, idf = lookup[word]
        values.append(count)
        indices.append(feature_index)
        weights.append(count * idf)
    pointers.append(len(values))
    index_array = np.asarray(indices, dtype=np.int64)
    pointer_array = np.asarray(pointers, dtype=np.int64)
    shape = (len(records), feature_count)
    X_counts = sparse.csr_matrix(
        (np.asarray(values, dtype=np.int32), index_array, pointer_array), shape=shape
    )
    X_tfidf = sparse.csr_matrix(
        (np.asarray(weights, dtype=np.float32), index_array.copy(), pointer_array.copy()),
        shape=shape
    )
    X_counts.sort_indices()
    X_tfidf.sort_indices()
    X_tfidf = normalize(X_tfidf, norm="l2", copy=False)
    return X_counts, X_tfidf

def write_feature_batches(run_dir, feature_count, batch_rows):
    if batch_rows < 1:
        raise ValueError("Feature batch size must be positive.")
    source_info = json.loads((run_dir / "preprocessing_status.json").read_text(encoding="utf-8"))
    vocab_info = json.loads((run_dir / "vocabulary_status.json").read_text(encoding="utf-8"))
    if (source_info["status"] != "complete" or vocab_info["status"] != "complete"
        or vocab_info["features"] != feature_count):
        raise RuntimeError("A completed matching preprocessing run and vocabulary are required.")
    # Use a fresh folder for each feature export, including reruns.
    feature_dir = run_dir / f"features_{uuid.uuid4().hex[:8]}"
    feature_dir.mkdir()
    manifest_path = feature_dir / "manifest.partial.jsonl"
    status_path = feature_dir / "status.json"
    write_json(status_path, {"status": "running"})
    connection = open_database(run_dir / "statistics.sqlite")
    rows_written = 0
    nonzero_count = 0
    part = 0
    start = time.perf_counter()
    peak_observed_ram = ram_gib()
    progress = display("Starting feature batches...", display_id=True)
    try:
        with manifest_path.open("w", encoding="utf-8") as manifest:
            for part, records in enumerate(
                iter_jsonl_batches(run_dir / "complaints.jsonl", batch_rows), start=part
            ):
                assert [record["matrix_row"] for record in records] == list(
                    range(rows_written, rows_written + len(records)))
                X_counts, X_tfidf = matrices_for_batch(records, connection, feature_count)
                assert X_counts.shape == X_tfidf.shape
                assert np.isfinite(X_tfidf.data).all()
                assert np.allclose(
                    np.asarray(X_tfidf.multiply(X_tfidf).sum(axis=1)).ravel(), 1,
                    rtol=1e-5, atol=1e-6)
                counts_name = f"counts_{part:06d}.npz"
                tfidf_name = f"tfidf_{part:06d}.npz"

```

```

sparse.save_npz(feature_dir / counts_name, X_counts)
sparse.save_npz(feature_dir / tfidf_name, X_tfidf)
manifest.write(json.dumps({
    "part": part, "row_start": rows_written,
    "row_end": rows_written + len(records), "rows": len(records),
    "features": feature_count, "nonzero": X_counts.nnz,
    "counts_file": counts_name, "tfidf_file": tfidf_name
})) + "\n")
manifest.flush()
rows_written += len(records)
nonzero_count += X_counts.nnz
memory = ram_gib()
peak_observed_ram = max(peak_observed_ram, memory)
message = f"Feature rows saved: {rows_written:,} | RSS ~{memory:.2f} GiB"
progress.update(message)
if part == 1 or part % PRINT_EVERY_CHUNKS == 0:
    print(message)
del records, X_counts, X_tfidf
gc.collect()
assert rows_written == source_info["stats"]["kept_rows"]
manifest_path.replace(feature_dir / "manifest.jsonl")
write_json(status_path, {"status": "complete", "rows": rows_written,
    "features": feature_count, "parts": part, "nonzero": nonzero_count,
    "elapsed_seconds": time.perf_counter() - start,
    "observed_peak_process_rss_gib": peak_observed_ram,
    "tfidf_dtype": "float32", "norm": "l2", "smooth_idf": True})
except BaseException as error:
    write_json(status_path, {"status": "incomplete", "rows_written": rows_written,
        "error": f"{type(error).__name__}: {error}"})
    raise
finally:
    connection.close()
print(f"Saved {part:,} sparse batches covering {rows_written:,} complaints.")
return feature_dir

```

```

FEATURE_DIR = write_feature_batches(RUN_DIR, FEATURE_COUNT, MATRIX_BATCH_ROWS)
feature_info = json.loads((FEATURE_DIR / "status.json").read_text(encoding="utf-8"))
display(pd.Series(feature_info, name="Feature export").to_frame())

```

'Feature rows saved: 1,292,206 | RSS ~0.26 GiB'

Feature rows saved:	1,000	RSS ~0.23 GiB
Feature rows saved:	20,000	RSS ~0.24 GiB
Feature rows saved:	40,000	RSS ~0.25 GiB
Feature rows saved:	60,000	RSS ~0.25 GiB
Feature rows saved:	80,000	RSS ~0.25 GiB
Feature rows saved:	100,000	RSS ~0.25 GiB
Feature rows saved:	120,000	RSS ~0.26 GiB
Feature rows saved:	140,000	RSS ~0.25 GiB
Feature rows saved:	160,000	RSS ~0.25 GiB
Feature rows saved:	180,000	RSS ~0.26 GiB
Feature rows saved:	200,000	RSS ~0.26 GiB
Feature rows saved:	220,000	RSS ~0.26 GiB
Feature rows saved:	240,000	RSS ~0.26 GiB
Feature rows saved:	260,000	RSS ~0.26 GiB
Feature rows saved:	280,000	RSS ~0.26 GiB
Feature rows saved:	300,000	RSS ~0.26 GiB
Feature rows saved:	320,000	RSS ~0.26 GiB
Feature rows saved:	340,000	RSS ~0.25 GiB
Feature rows saved:	360,000	RSS ~0.26 GiB
Feature rows saved:	380,000	RSS ~0.26 GiB
Feature rows saved:	400,000	RSS ~0.26 GiB
Feature rows saved:	420,000	RSS ~0.26 GiB
Feature rows saved:	440,000	RSS ~0.26 GiB
Feature rows saved:	460,000	RSS ~0.26 GiB
Feature rows saved:	480,000	RSS ~0.26 GiB
Feature rows saved:	500,000	RSS ~0.26 GiB
Feature rows saved:	520,000	RSS ~0.26 GiB
Feature rows saved:	540,000	RSS ~0.26 GiB
Feature rows saved:	560,000	RSS ~0.26 GiB
Feature rows saved:	580,000	RSS ~0.26 GiB
Feature rows saved:	600,000	RSS ~0.26 GiB
Feature rows saved:	620,000	RSS ~0.26 GiB
Feature rows saved:	640,000	RSS ~0.26 GiB
Feature rows saved:	660,000	RSS ~0.26 GiB
Feature rows saved:	680,000	RSS ~0.26 GiB
Feature rows saved:	700,000	RSS ~0.26 GiB
Feature rows saved:	720,000	RSS ~0.26 GiB
Feature rows saved:	740,000	RSS ~0.26 GiB
Feature rows saved:	760,000	RSS ~0.26 GiB
Feature rows saved:	780,000	RSS ~0.26 GiB
Feature rows saved:	800,000	RSS ~0.25 GiB
Feature rows saved:	820,000	RSS ~0.25 GiB
Feature rows saved:	840,000	RSS ~0.26 GiB
Feature rows saved:	860,000	RSS ~0.26 GiB
Feature rows saved:	880,000	RSS ~0.26 GiB
Feature rows saved:	900,000	RSS ~0.26 GiB
Feature rows saved:	920,000	RSS ~0.26 GiB
Feature rows saved:	940,000	RSS ~0.26 GiB
Feature rows saved:	960,000	RSS ~0.26 GiB
Feature rows saved:	980,000	RSS ~0.26 GiB
Feature rows saved:	1,000,000	RSS ~0.26 GiB
Feature rows saved:	1,020,000	RSS ~0.26 GiB
Feature rows saved:	1,040,000	RSS ~0.26 GiB
Feature rows saved:	1,060,000	RSS ~0.26 GiB
Feature rows saved:	1,080,000	RSS ~0.26 GiB
Feature rows saved:	1,100,000	RSS ~0.26 GiB

```

Feature rows saved: 1,120,000 | RSS ~0.26 GiB
Feature rows saved: 1,140,000 | RSS ~0.26 GiB
Feature rows saved: 1,160,000 | RSS ~0.25 GiB
Feature rows saved: 1,180,000 | RSS ~0.26 GiB
Feature rows saved: 1,200,000 | RSS ~0.26 GiB
Feature rows saved: 1,220,000 | RSS ~0.26 GiB
Feature rows saved: 1,240,000 | RSS ~0.26 GiB
Feature rows saved: 1,260,000 | RSS ~0.26 GiB
Feature rows saved: 1,280,000 | RSS ~0.26 GiB
Saved 1,293 sparse batches covering 1,292,206 complaints.

```

Feature export	
status	complete
rows	1292206
features	168291
parts	1293
nonzero	100293156
elapsed_seconds	449.364335
observed_peak_process_rss_gib	0.270302
tfidf_dtype	float32
norm	l2
smooth_idf	True

9. Show a small count and TF-IDF comparison

Here we load only the first sparse batch and display five rows and ten words. We then delete the batch matrices. The preview does not load the full dataset.

```

In [12]: with (FEATURE_DIR / "manifest.jsonl").open(encoding="utf-8") as handle:
          first_part = json.loads(next(handle))
          with closing(open_database(RUN_DIR / "statistics.sqlite")) as connection:
              preview_words = connection.execute(
                  "SELECT feature_index, term FROM features ORDER BY occurrences DESC, term L
              ).fetchall()
          preview_indices = [row[0] for row in preview_words]
          preview_labels = [row[1] for row in preview_words]
          first_counts = sparse.load_npz(FEATURE_DIR / first_part["counts_file"])
          first_tfidf = sparse.load_npz(FEATURE_DIR / first_part["tfidf_file"])
          print("Word counts (first five retained complaints):")
          display(pd.DataFrame(first_counts[:5, preview_indices].toarray(), columns=preview_l
          print("TF-IDF using global IDF, for the same rows and words:")
          display(pd.DataFrame(first_tfidf[:5, preview_indices].toarray(), columns=preview_la
          print("Full batch shape:", first_counts.shape)
          del first_counts, first_tfidf, preview_indices, preview_labels, preview_words

```

```
gc.collect();
print("Preview matrices released. No full feature matrix was loaded.")
```

Word counts (first five retained complaints):

	the	be	i	...	of	have	that
0	2	12	6	...	1	0	0
1	1	0	8	...	1	5	5
2	3	2	3	...	2	0	3
3	33	15	4	...	33	5	12
4	13	9	11	...	2	3	7

5 rows × 10 columns

TF-IDF using global IDF, for the same rows and words:

	the	be	i	...	of	have	that
0	0.028	0.160	0.081	...	0.016	0.000	0.000
1	0.030	0.000	0.233	...	0.035	0.160	0.182
2	0.126	0.081	0.123	...	0.098	0.000	0.154
3	0.225	0.099	0.027	...	0.263	0.037	0.100
4	0.215	0.144	0.178	...	0.039	0.053	0.141

5 rows × 10 columns

Full batch shape: (1000, 168291)

Preview matrices released. No full feature matrix was loaded.

10. Check the results and read batches later

Here we show how many rows were removed and where files were saved. Counts and TF-IDF use exactly the same complaints and feature indices. Topic analysis can read batches with the generators below. Do not use `list(...)`, `pd.concat(...)` or `sparse.vstack(...)` over all batches unless you deliberately want the full data in RAM again.

JSONL keeps the original text for later embedding models when `KEEP_ORIGINAL_TEXT = True`. No automatic language filter is applied. The results describe submitted complaints, not a representative population survey. Later topic models may have their own memory needs.

```
In [13]: def iter_text_frames(run_dir=RUN_DIR, batch_rows=1_000):
          for records in iter_jsonl_batches(run_dir / "complaints.jsonl", batch_rows):
              yield pd.DataFrame.from_records(records)

          def iter_feature_matrices(kind="tfidf", feature_dir=FEATURE_DIR):
              if kind not in {"counts", "tfidf"}:
```

```

        raise ValueError("Choose counts or tfidf.")
    state = json.loads((feature_dir / "status.json").read_text(encoding="utf-8"))
    if state["status"] != "complete":
        raise RuntimeError("This feature export is incomplete.")
    with (feature_dir / "manifest.jsonl").open(encoding="utf-8") as handle:
        for line in handle:
            entry = json.loads(line)
            yield entry, sparse.load_npz(feature_dir / entry[f"{kind}_file"])

assert preprocessing_info["stats"]["kept_rows"] == feature_info["rows"]
with closing(open_database(RUN_DIR / "statistics.sqlite")) as connection:
    stop_word_check = pd.read_sql_query(
        "SELECT term, doc_freq, occurrences FROM features "
        "WHERE term IN ('the', 'and', 'a', 'i', 'not') ORDER BY term", connection)
print("Stop words found in the saved vocabulary (absent source words need not appear):")
display(stop_word_check)
del stop_word_check
print("Final retained complaints:", f"{feature_info['rows']:,}")
print("Features:", f"{FEATURE_COUNT:,}")
print("Text:", RUN_DIR / "complaints.jsonl")
print("Vocabulary:", RUN_DIR / "vocabulary.csv")
print("Feature manifest:", FEATURE_DIR / "manifest.jsonl")
print("Text-processing audit:", RUN_DIR / "preprocessing_status.json")
print("The variable df contains only five preview rows.")
print("Save this notebook with Ctrl+S before running the final cell.")

```

Stop words found in the saved vocabulary (absent source words need not appear):

	term	doc_freq	occurrences
0	a	1044335	4935592
1	and	1105598	6152781
2	i	1161348	7833378
3	not	910964	2420817
4	the	1132384	9598183

Final retained complaints: 1,292,206

Features: 168,291

Text: C:\Users\123wi\OneDrive\Desktop\duits uni\data analysis\preprocessing_streamed\run_20260923_103853_ac056b\complaints.jsonl

Vocabulary: C:\Users\123wi\OneDrive\Desktop\duits uni\data analysis\preprocessing_streamed\run_20260923_103853_ac056b\vocabulary.csv

Feature manifest: C:\Users\123wi\OneDrive\Desktop\duits uni\data analysis\preprocessing_streamed\run_20260923_103853_ac056b\features_2ee9a01c\manifest.jsonl

Text-processing audit: C:\Users\123wi\OneDrive\Desktop\duits uni\data analysis\preprocessing_streamed\run_20260923_103853_ac056b\preprocessing_status.json

The variable df contains only five preview rows.

Save this notebook with Ctrl+S before running the final cell.

Method references

- [Kaggle complaint dataset](#)

- pandas CSV chunks
- NLTK WordNet lemmatisation
- loky process workers
- scikit-learn TF-IDF

11. Export the notebook and outputs to PDF

Press Ctrl+S first, then run the final cell. The export uses the saved notebook and its existing outputs. It does not rerun preprocessing or embed the full JSONL file. Its own completion message appears after the PDF is created and is not included in it. If you renamed or moved this notebook, update `NOTEBOOK_PATH` in section 1.

```
In [ ]: # NOTEBOOK_PDF_EXPORT
import asyncio
import subprocess
import sys
import tempfile
from pathlib import Path

import nbformat

notebook_path = Path("complaints_preprocessing.ipynb")
output_dir = Path(OUTPUT_DIR)
output_dir.mkdir(parents=True, exist_ok=True)

input("Press Ctrl+S to save the notebook and its outputs, then press Enter here: ")

if not notebook_path.is_file():
    raise FileNotFoundError(
        f"Notebook not found: {notebook_path}\n"
        "Update NOTEBOOK_PATH to the actual notebook filename."
    )

notebook = nbformat.read(notebook_path, as_version=4)

# Clear previous PDF-export errors from the exported copy only.
for cell in notebook.cells:
    if cell.cell_type == "code" and (
        "pdf-export" in cell.metadata.get("tags", [])
        or "NOTEBOOK_PDF_EXPORT" in cell.source
        or "WebPDFExporter" in cell.source
    ):
        cell.outputs = []
        cell.execution_count = None

# Unexecuted optional cells do not prevent exporting saved outputs.
if not any(
    cell.get("outputs")
    for cell in notebook.cells
    if cell.cell_type == "code"
):
    raise RuntimeError(
```

```

        "The saved notebook contains no outputs. "
        "Check NOTEBOOK_PATH, save with Ctrl+S, and run this cell again."
    )

notebook.cells.insert(
    0,
    nbformat.v4.new_raw_cell(
        "<style>"
        "pre, code {white-space: pre-wrap !important; "
        "overflow-wrap: anywhere !important;}"
        ".jp-OutputArea-output, .jp-RenderedText "
        "{overflow: visible !important;}"
        "table {font-size: 10px !important;}"
        "</style>",
        metadata={"raw_mimetype": "text/html"},
    ),
)

pdf_path = output_dir / f"{notebook_path.stem}.pdf"

export_script = """
import asyncio
import sys
from pathlib import Path
from nbconvert import WebPDFExporter

if sys.platform == "win32":
    asyncio.set_event_loop_policy(
        asyncio.WindowsProactorEventLoopPolicy()
    )

exporter = WebPDFExporter()
exporter.allow_chromium_download = True
exporter.paginate = True
exporter.exclude_input = False
exporter.exclude_output = False

pdf_bytes, _ = exporter.from_filename(sys.argv[1])
Path(sys.argv[2]).write_bytes(pdf_bytes)
"""

print("Exporting the saved notebook and outputs without rerunning preprocessing...")

with tempfile.TemporaryDirectory(dir=str(output_dir)) as temporary_dir:
    snapshot_path = Path(temporary_dir) / notebook_path.name
    nbformat.write(notebook, snapshot_path)

    result = subprocess.run(
        [
            sys.executable,
            "-c",
            export_script,
            str(snapshot_path),
            str(pdf_path),
        ],
        capture_output=True,
    )

```

```

        text=True,
        encoding="utf-8",
        errors="replace",
    )

if result.returncode != 0:
    print(result.stdout[-4000:])
    print(result.stderr[-8000:])
    raise RuntimeError(
        "PDF export failed. Check the error above. "
        "If Chromium is missing, run:\n"
        "import subprocess, sys\n"
        'subprocess.check_call([sys.executable, "-m", '
        '"playwright", "install", "chromium"])'
    )

print("PDF saved:", pdf_path)
print(f"PDF size: {pdf_path.stat().st_size / 1024:.1f} KiB")

```

In []: